

LA SUITE DE COLLATZ : UNE CONJECTURE FORT CÉLÈBRE ! (AVEC C/GMP ET PYTHON/BOKEH)

PETER PHILIPPE

En 1928, c'est en toute discrétion que le mathématicien Lothar Collatz¹ proposa sa fameuse suite, d'apparence anodine et manipulable par tout le monde, elle continue de nos jours à faire couler beaucoup d'encre (et éteindre beaucoup de pixels). En partant d'une valeur entière $n > 0$, cette suite conjecture de toujours terminer par le cycle $\{4, 2, 1\}$ quelque soit la valeur n de départ. Voici ce petit algorithme fonctionnant par récurrence :

$$n = \begin{cases} n/2 & \text{si } n \equiv [2] \pmod{0} \text{ donc si } n \text{ est pair.} \\ 3n + 1 & \text{autrement.} \end{cases}$$

Le triplet $\{4, 2, 1\}$ se répète indéfiniment, car en partant du chiffre impair 1, on applique alors $3n + 1$ ce qui donne 4, ce dernier étant pair, il est donc divisé par 2 permettant d'obtenir 2, lui aussi pair, une division supplémentaire par 2 permet d'arriver à 1 et cela repart pour un petit tour, c'est enfantin. Par exemple, la suite de Collatz du nombre 15 est la suivante :

la longueur du vol est de 17 et l'altitude maximale de 160
[46, 23, 70, 35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, **4, 2, 1**]

Cette suite semble donc assez simpliste au premier abord, mais elle cache en réalité une complexité sous-jacente qui n'a toujours pas été élucidée, il s'agit d'un « monstre » mathématique aux coulisses encore sous-explorées. Car depuis plusieurs décennies, des amateurs du monde entier (n'y trouvant d'ailleurs qu'illusion, déconvenue et frustration) ainsi que des mathématiciens professionnels de tous les continents, s'attellent à tenter de percer le secret de cette conjecture, une beauté mystérieuse aux difficultés insoupçonnées.

Deux petits programmes sont proposés, le premier en langage C avec la librairie [GMP](#) et le second en langage Python avec la librairie [Bokeh](#) (pour la visualisation), ils permettent de s'amuser un peu avec cette suite. Ce sont mes tous premiers essais avec ces deux librairies que je découvre, ces petits programmes sont donc rudimentaires. Les codes sources et ce fichier PDF sont disponibles dans mon [compte github](#).

Enfin, je conseille notamment de lire le [cours universitaire](#) de Jean-Paul Delahaye et de Christian Lasou, ainsi que la lecture de trois articles scientifiques respectivement écrits par [Luc-Olivier Pochon et Alain Favre](#), [Gerhard Opfer](#) et [Jeffrey Clark Lagarias](#).

1. Lothar Collatz (1910-1990) - Mathématicien allemand

1. Langage C

Ce premier programme réclame comme arguments deux nombres entiers strictement positifs a et b (avec $a < b$). Attention toutefois à ne pas choisir un intervalle fermé de longueur « démesurée » telle que $[2^{24}; 2^{25}]$, car cela risquera d'être long avec un petit CPU.

Il y aura deux tests intéressants à effectuer, car ils permettent de trouver deux très grands nombres situés à une altitude maximale élevée (que l'on dénomme un **pic**). Ces deux nombres, 49163256101584231 et 2358909599867980429759 sont tirés du [site](#) de David Barina, les deux commandes qui suivent permettront de les calculer :

```
./collatzgmp 49163256101500000 49163256101600000
```

```
./collatzgmp 2358909599867980400000 2358909599867980500000
```

Il est d'ailleurs conseillé de consulter, entre autres, ce [site](#)² et [celui-là](#) de Eric Roosendaal.

Voici donc le code source du fichier `collatzgmp.c` :

```
#include <stdio.h>
#include <stdlib.h>
#include <gmp.h>

static void collatz_sequence(const mpz_t start, mpz_t summit) {
    mpz_t n, nn;

    mpz_init(nn);
    mpz_init_set(n, start);
    mpz_set(summit, start);

    while (mpz_cmp_ui(n, 1) != 0) {
        if (mpz_even_p(n)) {
            mpz_tdiv_q_2exp(n, n, 1); /* n = n / 2 */
        } else {
            mpz_mul_ui(nn, n, 3); /* n = 3 * n */
            mpz_add_ui(n, nn, 1); /* n = n + 1 */
        }
        if (mpz_cmp(n, summit) > 0) /* n > summit */
            mpz_set(summit, n);
    }

    mpz_clear(n);
    mpz_clear(nn);
}

static void find_altitude_maximale(const mpz_t a, const mpz_t b, mpz_t nMax, mpz_t altMax) {
    mpz_t n, alt;

    mpz_init_set(n, a);
    mpz_init(alt);
    mpz_set(nMax, a);
    mpz_set_ui(altMax, 0);
```

2. L'encylopédie en ligne des suites de nombres entiers.

LA SUITE DE COLLATZ : UNE CONJECTURE FORT CÉLÈBRE ! (AVEC C/GMP ET PYTHON/BOKEH)

```
while (mpz_cmp(n, b) <= 0) {
    collatz_sequence(n, alt);
    if (mpz_cmp(alt, altMax) > 0) {
        mpz_set(altMax, alt);
        mpz_set(nMax, n);
    }
    mpz_add_ui(n, n, 1); /* n += 1 */
}
mpz_clear(alt);
mpz_clear(n);
}

int main(int argc, char **argv) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <a> <b> (a < b, strictly positive integers)\n", argv[0]);
        return EXIT_FAILURE;
    }

    mpz_t a, b;
    mpz_init(a);
    mpz_init(b);

    if (mpz_set_str(a, argv[1], 10) != 0 || mpz_set_str(b, argv[2], 10) != 0) {
        fprintf(stderr, "Invalid input(s)!\n");
        mpz_clear(b);
        mpz_clear(a);
        return EXIT_FAILURE;
    }

    if (mpz_sgn(a) <= 0 || mpz_sgn(b) <= 0 || mpz_cmp(a, b) > 0) {
        fprintf(stderr, "A and B must be strictly positive integers such that a < b.\n");
        mpz_clear(b);
        mpz_clear(a);
        return EXIT_FAILURE;
    }

    mpz_t n, altMax;
    mpz_init(n);
    mpz_init(altMax);

    find_altitude_maximale(a, b, n, altMax);

    gmp_printf("\n\tIn [%Zd, %Zd], the maximal altitude '%Zd'\n\tis obtained with the number %Zd.\n\n", a, b, altMax, n);

    mpz_clear(altMax);
    mpz_clear(n);
    mpz_clear(b);
    mpz_clear(a);

    return EXIT_SUCCESS;
}
```

```
% ./collatzgmp 1000 2000
```

```
In [1000, 2000], the maximal altitude '1276936'
is obtained with the number 1819.
```

2. Langage Python

Là aussi, il sera intéressant de commencer par essayer avec les deux intervalles proposés pour le code source en langage C, à savoir [49163256101500000;49163256101600000] et [2358909599867980400000;2358909599867980500000] où l'on observera à chaque fois un « pic » très impressionnant.

Et voici donc le second code source permettant de visualiser la suite et le pic (il faudra s'assurer d'avoir la librairie Bokeh d'installée), son exécution se fait dans un terminal via `bokeh serve -show collatzbokeh.py` :

```
from bokeh.plotting import figure, curdoc
from bokeh.models import ColumnDataSource, Div, Button, TextInput, Span, Spacer
from bokeh.layouts import import column, row

def del_comma(x: int) -> str:
    return f"{x:,}".replace(",", "")

def collatz_max_altitude(n: int) -> int:
    peak = n
    x = n
    while x != 1:
        x = x >> 1 if x & 1 == 0 else 3 * x + 1
        if x > peak:
            peak = x
    return peak

def search_max_altitude(a: int, b: int) -> tuple[int, int]:
    max_n, max_alt = a, 0
    for n in range(a, b + 1):
        alt = collatz_max_altitude(n)
        if alt > max_alt:
            max_n, max_alt = n, alt
    return max_n, max_alt

def collatz(n: int) -> list:
    seq = [n]
    while n != 1:
        n = n >> 1 if n % 2 == 0 else 3 * n + 1
        seq.append(n)
    return seq

curdoc().theme = "dark_minimal"

p = figure(
    height=500, width=1000,
    x_axis_label = "Steps",
    y_axis_label = "Value (log scale)",
    y_axis_type="log",
    tools = "pan,wheel_zoom,box_zoom,save, reset",
    toolbar_location="above",
)

source = ColumnDataSource(data=dict(x=[], y=[]))
source_marker = ColumnDataSource(data=dict(x=[], y=[]))
r_line = p.line("x", "y", source=source, line_width=1.5, alpha=1.0)
r_circles = p.scatter("x", "y", source=source, size=4, alpha=0.5)
```

LA SUITE DE COLLATZ : UNE CONJECTURE FORT CÉLÈBRE ! (AVEC C/GMP ET PYTHON/BOKEH)

```

r_peak = p.scatter("x", "y",
                   source=source_marker, size=12, color="#ff0000",
                   marker="star_dot", line_color="#ff0000",
                   legend_label="Maximum altitude")

p.add_layout(Span(location=1, dimension='width',
                  line_color='aaaaaa',
                  line_dash='dashed',
                  line_width=1))
p.legend.location = "top_right"
p.legend.label_text_font_size = "11px"

title_div = Div(
    text = "<h2 style='margin:0;font-family:sans-serif;color:#20262b'>"
           "Collatz: maximum altitude on [a,b]</h2>", width=1000
)

input_a = TextInput(title="", value="1", width=200)
input_b = TextInput(title="", value="1000", width=200)
row_a = row(Div(text="[a]", width=8, styles={"line-height": "35px"}), input_a)
row_b = row(Div(text="[b]", width=8, styles={"line-height": "35px"}), input_b)
inputs = row(row_a, row_b)

button_search = Button(label="Search", width=150, height=30, button_type="success")

check_div = Div(
    text = "", width = 1000,
    styles = {"font-family": "monospace", "font-size": "13px"}
)

result_div = Div(
    text = "", width = 1000,
    styles = {
        "font-family": "monospace", "font-size": "13px",
        "background": "#d4ddff", "padding": "10px",
        "border-radius": "6px", "border": "2px solid #20262b",
        "min-height": "36px"
    }
)

def search() -> None:
    try:
        a = int(input_a.value.strip())
        b = int(input_b.value.strip())
        assert 1 <= a < b
    except (ValueError, AssertionError):
        check_div.text = "<span style='color:#ff0000'>A < B and must be greater than 0.</span>"
        return

    if b - a > 10**6:
        check_div.text = "<span style='color:#ff0000'>Interval too large (max 10^6).</span>"
        return

    max_n, max_alt = search_max_altitude(a, b)
    sequence = collatz(max_n)
    peak_step = sequence.index(max_alt)
    source.data = dict(x=list(range(len(sequence))), y=sequence)
    source_marker.data = dict(x=[peak_step], y=[max_alt])

```

```

r_line.glyph.line_color = "mediumslateblue"
r_circles.glyph.fill_color = "skyblue"
r_circles.glyph.line_color = "skyblue"

p.title.text = (
    f"Collatz Sequence, N = {del_comma(max_n)}, "
    f"({del_comma(len(sequence)-1)} steps, Max Altitude = {del_comma(max_alt)})"
)

result_div.text = (
    f"<b>Interval</b> : [{del_comma(a)}, {del_comma(b)}], &nbsp;&nbsp;&nbsp;"
    f"<b>Number</b> : {del_comma(max_n)}, &nbsp;&nbsp;&nbsp;"
    f"<b>Max Altitude</b> : {del_comma(max_alt)}, &nbsp;&nbsp;&nbsp;"
    f"<b>Steps</b> : {del_comma(len(sequence)-1)}, &nbsp;&nbsp;&nbsp;"
    f"<b>Steps (peak)</b> : {del_comma(peak_step)}"
)

button_search.on_click(lambda: search())
controls = row(Spacer(width=50), inputs, Spacer(height=20), button_search, align="start")
curdoc().add_root(column(title_div, controls, Spacer(width=50), check_div, p, result_div,
                        sizing_mode="fixed"))
curdoc().title = "Collatz Max Altitude"

```

Collatz: maximum altitude on [a,b]

[a b]

